

SOCRATES: An AI-Powered SOC Agent for Automated Network Alert Triage

Abstract—Security Operations Centers (SOCs) face severe alert fatigue as modern security systems generate massive volumes of redundant and false-positive alerts that exceed analysts’ investigation capacity. Traditional rule-based and learning-based approaches can reduce part of this workload, but remain limited by rigid heuristics, insufficient contextual modeling, and poor adaptability to evolving enterprise environments. Large Language Models (LLMs) offer strong semantic understanding and reasoning capabilities for alert investigation, yet directly applying them to operational alert streams remains impractical due to limited inference throughput and a tendency to conservatively escalate context-poor benign alerts.

Motivated by an empirical study of operational alert streams, we propose SOCRATES, an AI-powered SOC agent that performs scalable alert triage through a coarse-to-fine pipeline. First, SOCRATES normalizes heterogeneous alerts, aggregates predefined high-frequency behaviors into traceable meta-alerts, and filters recurring benign activities using a Hierarchical Alert Event Tree (HAT) learned from analyst-confirmed benign examples. Second, it constructs a heterogeneous Alert-Entity Graph (AEG) and prioritizes the remaining alerts according to the rarity of their patterns, participating entities, and relations. Third, it retrieves enterprise-specific false-positive knowledge and reconstructs backward and forward behavioral context from the graph, enabling an LLM to produce evidence-grounded and interpretable adjudication results.

We evaluate SOCRATES primarily on the public AIT-ADS benchmark. On 2,477,576 test alerts, SOCRATES reduces the analyst-facing alert stream to only 709 alerts, achieving a 99.9714% alert reduction ratio while retaining 99.9970% of attack alerts, with an FPR of 0.6434% and an F1-score of 99.8637%. SOCRATES also demonstrates a stronger overall balance of attack preservation, triage accuracy, and alert reduction than representative prior approaches. Moreover, it has been deployed in a real-world enterprise SOC for more than one year, demonstrating the practicality of combining lightweight alert reduction with evidence-driven LLM investigation at operational scale.

I. INTRODUCTION

With the increasing sophistication and automation of cyberattacks, modern enterprises are facing an ever-growing and increasingly complex threat landscape. To effectively detect and mitigate malicious activities, organizations have deployed a wide range of security devices, including Web Application Firewalls (WAFs) [30], Intrusion Detection Systems (IDSs) [5], and Intrusion Prevention Systems (IPSs) [27]. While these

security devices significantly enhance defensive visibility, they paradoxically give rise to a severe problem known as **alert fatigue** [4], [14], [17], [36]. In large-scale enterprise environments, Security Operations Centers (SOCs) routinely receive millions of alerts from these devices each day, a volume that far exceeds the processing capacity of human analysts; over 99% of these alerts are ultimately identified as false positives [1], [2], [45]. To alleviate this burden, industry practices have extensively adopted heuristic-based detection techniques as well as traditional machine learning and deep learning paradigms [3], [37], [39]. However, these approaches remain constrained by inherent limitations [34]. Rule-based systems are rigid and costly to maintain, requiring continuous manual updates to keep pace with evolving attack patterns [41]. Data-driven learning models, on the other hand, often operate as black boxes, lacking the interpretability necessary for analysts to trust their decisions. Moreover, such models typically fail to capture the deep semantic structure of alert logs or to correlate attacker behaviors across contextual dimensions, resulting in missed detections of sophisticated and stealthy threats.

Recently, Large Language Models (LLMs) have demonstrated remarkable capabilities in semantic understanding and contextual reasoning [15], [20], enabling them to interpret complex data with a level of flexibility previously unattainable by traditional learning models. Their ability to process heterogeneous inputs and reason over long textual contexts makes them especially attractive for security analysis tasks [32], where alerts often contain rich but fragmented semantic information distributed across logs, payloads, and metadata. Beyond general natural language tasks, LLMs have already shown promising results across multiple cybersecurity domains [44], such as automated penetration testing [8], [33], vulnerability discovery and analysis [24], [29], [46], and security policy auditing. LLMs have been increasingly explored as a potential paradigm for automating security workflows and bridging the gap between low-level alert data and high-level human reasoning.

Research Gaps. Despite the significant potential of LLMs, directly applying them to real-world alert classification and triage remains several gaps:

- ① **Throughput Gap.** In large enterprise environments, SOCs can receive millions of alerts each day, whereas LLMs have limited inference throughput and incur substantial computational overhead when analyzing individual alerts. Processing every incoming alert independently with an LLM would therefore create a severe bottleneck, causing investigation

backlogs and delaying responses to time-sensitive threats. Moreover, routine and redundant alerts would consume the same scarce inference capacity as security-critical cases. A practical LLM-based SOC system must thus reduce and prioritize the alert stream before invoking expensive LLM reasoning, while ensuring that genuine attacks are not discarded during early-stage processing.

- ② **False-Positive Gap.** Direct or lightweight LLM-based alert triage can exhibit a conservative decision bias: when an alert contains suspicious semantics but lacks sufficient context, the model may escalate it rather than risk overlooking an attack. Although this behavior can preserve a high attack detection rate, it can also cause benign activities that resemble malicious behavior to be classified as genuine threats. Recent evaluations expose this limitation. Cortex reports false-positive rates of 29.8% for prompt-only single-agent triage and 24.9% even when tool use is enabled, while SIABench finds that lightweight models can be biased toward treating alerts as genuine threats [16], [42]. Consequently, directly applying an LLM does not necessarily eliminate the large volume of false positives in operational alert streams; instead, incorrectly escalated benign alerts continue to accumulate and consume analyst attention. The key challenge is therefore to reduce false-positive decisions without weakening the system’s ability to retain genuine attacks.

Our Approach. To address these gaps, we propose SOCRATES, an AI-powered SOC agent for scalable and evidence-grounded network alert triage. We first conduct an empirical study of operational security alerts to ground the system design in the characteristics of real-world alert streams.

Our key observation is threefold: (1) security alerts are highly repetitive, with a single attack often triggering many recurring low-level alerts; (2) attack context differs fundamentally from an LLM’s sequential context window: relevant alerts are logically connected but may be scattered across the alert stream; and (3) contextual knowledge is vast but unevenly informative, with only a small subset providing strong attack indicators.

Building on these observations, we design SOCRATES as a staged, coarse-to-fine alert-triage pipeline. First, SOCRATES normalizes heterogeneous alerts, consolidates predefined high-frequency behaviors into traceable meta-alerts, and filters recurring benign activities using a Hierarchical Alert Event Tree (HAT) learned from analyst-confirmed benign examples. Second, it constructs a directed heterogeneous Alert-Entity Graph (AEG) over the remaining alerts and meta-alerts, and prioritizes them according to the rarity of their alert patterns, participating entities, and relations, filtering low-priority candidates before LLM investigation. Finally, for the remaining ambiguous alerts, SOCRATES retrieves enterprise-specific false-positive knowledge and reconstructs backward and forward behavioral context from the AEG. The LLM then follows a structured evidence-guided process to interpret the alert, inspect its behavioral context, test the applicability of

retrieved benign patterns, and produce a traceable True Alert or False Positive verdict.

Technical Challenges. Building such a system requires addressing three technical challenges.

- ① **How can recurring benign alerts be recognized without suppressing genuine attacks?** Benign business activities often produce alerts with stable underlying structures, while instance-specific fields, such as identifiers, tokens, addresses, and request parameters, vary across individual events. Operational SOC research shows that benign triggers account for a substantial share of alerts, while analysts report that false-positive alarms often stem from legitimate organizational behavior and still require manual validation [1], [45]. Exact matching therefore fails to recognize many repeated benign behaviors, whereas overly broad matching may incorrectly generalize an established benign pattern to a genuine attack. The challenge is to abstract dynamic variations while retaining the security-relevant attributes needed to distinguish benign repetitions from superficially similar malicious activities.
- ② **How can the context of an incomplete alert be reconstructed from temporally and relationally adjacent events?** An individual alert often captures only a fragment of the underlying activity and may omit the evidence needed to infer its intent or outcome. Security alerts exhibit both temporal locality, because related behaviors often occur within a bounded time interval, and relational locality, because events involving the same source, target, asset, account, or service may belong to the same behavioral progression. This need for contextual and provenance-aware security-event analysis has also been recognized in prior work [14], [39]. Simply collecting all nearby events, however, can introduce substantial coincidental and repetitive noise. The challenge is therefore to identify a compact set of genuinely relevant preceding and subsequent events and organize them into a coherent alert context without losing critical behavioral transitions.
- ③ **How can LLM decisions be grounded in sufficient and relevant security evidence?** A single alert rarely provides enough information to determine whether the underlying activity is malicious or benign. Similar payloads may arise from either an attack or a legitimate enterprise workflow, while the evidence needed for adjudication is often distributed across historical cases and preceding or subsequent events. Recent benchmarks show that cyber investigations require multi-step evidence across heterogeneous logs [16], [43]; Cortex further reports that context-poor, single-model alert analysis struggles with noisy enterprise data [42]. Providing all available context is also impractical because irrelevant or redundant information can obscure critical evidence. The challenge is to identify and organize the most relevant enterprise-specific and behavioral context so that the LLM can produce a reliable verdict supported by concrete, traceable observations.

Key Insights. To address these challenges, SOCRATES fol-

lows a coarse-to-fine design that progressively reduces alert redundancy, prioritizes unresolved alerts, and applies evidence-grounded reasoning only to the remaining ambiguous cases.

- **Hierarchical Benign Fingerprint Filtering.** The first module performs low-cost early reduction by normalizing heterogeneous alerts, consolidating predefined high-frequency behaviors into traceable meta-alerts, and filtering recurring benign activities through a Hierarchical Alert Event Tree (HAT). The HAT learns branch-specific fingerprints from analyst-confirmed benign alerts, preserving stable security-relevant attributes while abstracting recurring dynamic variations with typed and structural constraints. An alert is filtered only when it uniquely matches a valid benign fingerprint; otherwise, it is conservatively retained for downstream analysis.
- **Graph-Based Alert Prioritization.** The second module constructs a directed heterogeneous Alert-Entity Graph (AEG) over the remaining alerts, meta-alerts, and their associated entities. It prioritizes each alert according to the rarity of its alert pattern, participating entities, and complete entity-alert relation. High-frequency behaviors already consolidated into meta-alerts are counted as single occurrences, preventing burst size from dominating the graph statistics. Low-priority alerts are filtered, while anomalous candidates are retained for deeper investigation.
- **Evidence-Driven LLM Investigation.** The third module investigates the remaining ambiguous alerts using enterprise-specific false-positive knowledge and graph-derived behavioral context. SOCRATES retrieves relevant benign patterns from a knowledge base built from analyst-confirmed false positives and reconstructs backward and forward behaviors through bidirectional traversal of the AEG. The LLM then follows a structured evidence-guided process to interpret the alert, inspect its behavioral context, evaluate the applicability of retrieved benign patterns, and search for conflicting evidence before producing a traceable True Alert or False Positive verdict.

Evaluation. We evaluate SOCRATES primarily on the public AIT-ADS benchmark, where it processes 2,477,576 test alerts and reduces them to only 709 analyst-facing alerts. This corresponds to an end-to-end alert reduction ratio of 99.9714%, while still retaining 99.9970% of attack alerts. SOCRATES further achieves an FPR of 0.6434% and an F1-score of 99.8637%, showing that substantial alert reduction can be achieved without materially sacrificing attack preservation or triage accuracy. Compared with representative state-of-the-art alert-triage approaches evaluated on AIT-ADS, SOCRATES achieves a stronger overall balance among attack preservation, triage accuracy, and alert reduction. Our ablation results further confirm that aggregation, HAT filtering, graph-based prioritization, behavioral context, false-positive knowledge, and structured LLM reasoning each contribute to the effectiveness of the overall pipeline. In addition, SOCRATES has been deployed in a real-world enterprise environment for more than one year, demonstrating its practical applicability in

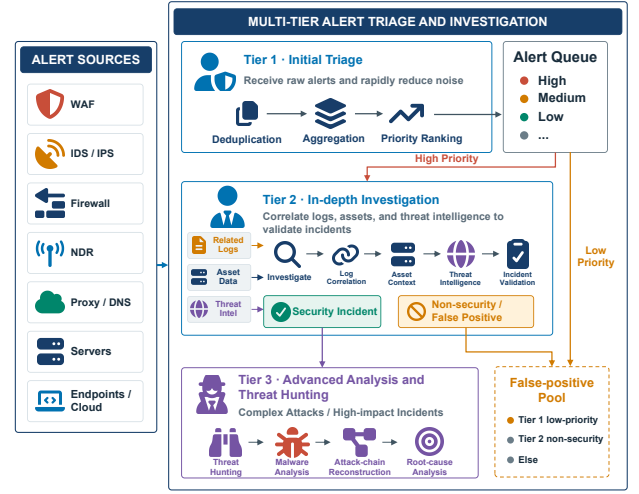


Fig. 1: Overview of SOC alert triage process.

operational SOC settings.

II. BACKGROUND & RELATED WORK

A. Alert triage in SOCs

As illustrated in Figure 1, network alert triage is typically organized as a tiered workflow to reduce alert noise and allocate analyst effort according to alert complexity and potential impact. Tier-1 analysts serve as the first line of review: they inspect the immediately available alert fields, payload snippets, rule descriptions, and basic contextual information to dismiss obvious false positives, assign an initial severity, and decide whether an alert can be closed or requires escalation. Alerts that remain ambiguous or appear suspicious are forwarded to Tier-2 analysts, who perform more detailed investigation by correlating related alerts and logs, examining asset and user context, checking historical behaviors, and consulting vulnerability information or threat intelligence to determine whether the activity indicates a real attack. When an alert involves high-impact assets, persistent adversarial behavior, or signs of compromise, Tier-3 analysts conduct advanced analysis and response, such as forensic investigation, malware analysis, threat hunting, root-cause analysis, containment, remediation, and refinement of detection rules.

B. LLM Applications in Cybersecurity

Most LLMs are built on the transformer architecture [40]. The key innovation of this architecture lies in the self-attention mechanism, which allows the model to evaluate the relative importance of different words in the input text and capture long-range dependencies and complex contextual relationships. These models are pre-trained on massive and diverse text corpora to learn the task of predicting the next word in a sequence. As a result, they develop a strong ability to understand meaning in language [47]. This emerging capacity for deep semantic understanding enables LLMs to interpret the implications and intentions behind semi-structured texts, such as the payloads in network alerts. The powerful text understanding and reasoning capabilities of LLMs have catalyzed

a wave of innovation within the cybersecurity domain [10], [44]. Beyond simple text classification, LLMs are now being explored for a wide range of complex cybersecurity tasks, including threat intelligence analysis [28], [35], malware code dissection [9], [18], penetration testing [8], [33], and incident response plan formulation [23]. Recognizing this potential, several cybersecurity and technology companies have begun developing their own security-specific LLMs. For instance, Microsoft has introduced Security Copilot [31], Google has developed Sec-PaLM as part of its Security AI Workbench [13], and CrowdStrike has launched Charlotte AI [6]. These models are typically trained on massive proprietary cybersecurity datasets and provide more accurate security analysis.

III. EMPIRICAL STUDY

To ground the design of SOCRATES in the characteristics of security alert streams, we conduct an empirical study using alerts from a production enterprise SOC together with attack annotations from AIT-ADS. Our analysis reveals three key observations: (1) alert streams are highly repetitive at both structural and behavioral levels; (2) attack context is logically connected rather than sequentially contiguous, with relevant alerts concentrated in local temporal and relational neighborhoods; and (3) available contextual knowledge is abundant but unevenly informative, with only a small subset providing decisive evidence for alert adjudication. These observations motivate the coarse-to-fine design of SOCRATES.

A. Observation 1: Alert streams are highly repetitive at both structural and behavioral levels

Operational alert streams exhibit substantial redundancy in two complementary forms. First, many benign alerts repeatedly instantiate the same underlying structure despite variations in instance-specific fields. As illustrated in Figure 2, four false-positive SQL-injection alerts share the same SQL statement, referenced tables, join structure, and parameter position, while differing only in the value of `S_USER_ID`. This pattern is also reflected at scale: as shown in Figure 3, clustering 5,138 benign SQL-injection alerts yields only 16 concentrated clusters, with the three largest accounting for 53.7% of all samples. These results indicate that many false positives are recurring manifestations of a limited set of stable request structures that cannot be captured by exact raw-string matching.

Second, alert streams also exhibit strong behavioral repetition, where a single underlying activity can trigger many recurring low-level alerts. As shown in Figure 4, several hundred thousand alerts are generated each day, with a small number of scanning- and brute-force-related alert types contributing a substantial fraction of the total volume, while confirmed true attacks remain comparatively sparse. Processing these repetitive alerts independently would waste downstream capacity and allow burst size to distort priority estimation. These observations motivate SOCRATES to abstract recurring structural variations through hierarchical benign fingerprints and

```
sqlstr=select+SYS_T_ROLE.S_ROLE_NAME%2cSYS_T_ROLE.
S_ROLE_REMARK%2cSYS_T_ROLE.S_ROLE_NO+from+SYS_T_ROLE
+inner+join+SYS_T_USERROLE+on+SYS_T_ROLE.S_ROLE_ID%3
dSYS_T_USERROLE.S_ROLE_ID+and+SYS_T_USERROLE.
S_USER_ID%3d' 8a0483e4991f745e019927daa58b02a9'
sqlstr=select+SYS_T_ROLE.S_ROLE_NAME%2cSYS_T_ROLE.
S_ROLE_REMARK%2cSYS_T_ROLE.S_ROLE_NO+from+SYS_T_ROLE
+inner+join+SYS_T_USERROLE+on+SYS_T_ROLE.S_ROLE_ID%3
dSYS_T_USERROLE.S_ROLE_ID+and+SYS_T_USERROLE.
S_USER_ID%3d' 8a9085b87fc7185c017fcf76dd3b4d8d'
sqlstr=select+SYS_T_ROLE.S_ROLE_NAME%2cSYS_T_ROLE.
S_ROLE_REMARK%2cSYS_T_ROLE.S_ROLE_NO+from+SYS_T_ROLE
+inner+join+SYS_T_USERROLE+on+SYS_T_ROLE.S_ROLE_ID%3
dSYS_T_USERROLE.S_ROLE_ID+and+SYS_T_USERROLE.
S_USER_ID%3d' 8a9085b73d35b53e013d43ecb2933c1e'
sqlstr=select+SYS_T_ROLE.S_ROLE_NAME%2cSYS_T_ROLE.
S_ROLE_REMARK%2cSYS_T_ROLE.S_ROLE_NO+from+SYS_T_ROLE
+inner+join+SYS_T_USERROLE+on+SYS_T_ROLE.S_ROLE_ID%3
dSYS_T_USERROLE.S_ROLE_ID+and+SYS_T_USERROLE.
S_USER_ID%3d' 8a8781c28389a1fb0183b647b48f04a8'
```

Fig. 2: Four false-positive SQL-injection alerts with identical query structures. Red text highlights the varying `S_USER_ID` values.

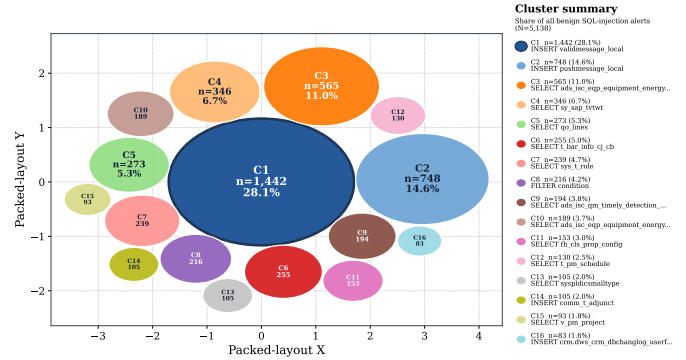


Fig. 3: Payload-similarity clustering of 5,138 benign SQL-injection alerts.

consolidate high-frequency behaviors into meta-alerts before subsequent analysis.

B. Observation 2: Attack context is logically connected rather than sequentially contiguous

Unlike the sequential context consumed by an LLM, security-relevant context is determined by behavioral relationships among alerts rather than their positions in the raw alert stream. Alerts belonging to the same attack activity may be interleaved with unrelated events, yet remain connected through shared entities and occur within bounded temporal intervals. To characterize this property, we use the *russellmitchell* scenario of AIT-ADS and treat alerts belonging to the same annotated attack phase as ground-truth related context. For each attack alert, we measure the fraction of its relevant context reachable within a temporal window Δ and an Alert-Entity Graph (AEG) search depth h . Specifically, $\text{Coverage}(\Delta, h)$ denotes the average fraction of related alerts satisfying both $|t_b - t_a| \leq \Delta$ and $d_G(a, b) \leq h$, where $d_G(\cdot, \cdot)$ is the shortest-path distance between two alert nodes in the AEG. Figure 5 shows the resulting coverage under different combinations of Δ and h .

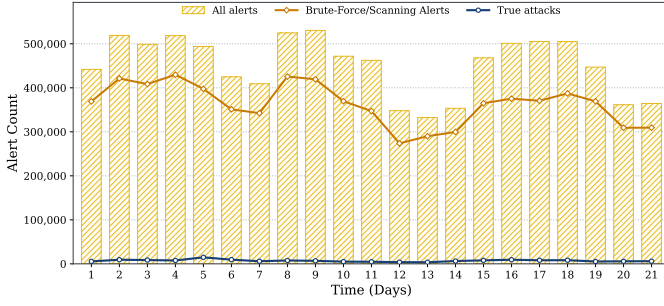


Fig. 4: Daily volumes of all alerts, the five dominant scanning- and brute-force-related alert types, and confirmed true attacks over 21 consecutive days.

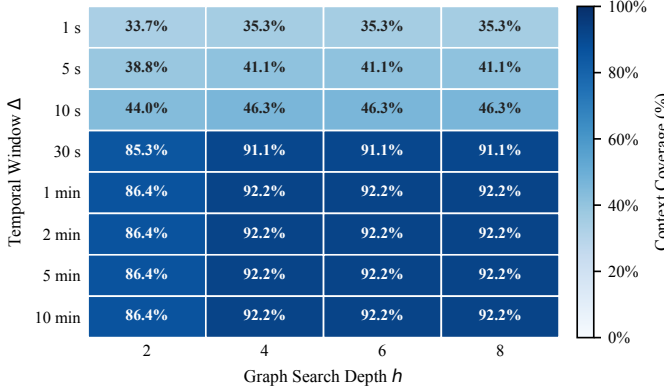


Fig. 5: Coverage of ground-truth attack-related context under different temporal windows Δ and AEG search depths h in the *russellmitchell* scenario of AIT-ADS. Each cell reports the fraction of relevant context satisfying both the temporal and relational constraints.

As shown in Figure 5, the relevant context is highly concentrated in a compact temporal-relational neighborhood. With a 30-second window, an AEG depth of $h = 2$ already covers 85.3% of the related context, while $h = 4$ increases the coverage to 91.1%. Expanding the window to one minute with $h = 4$ reaches 92.2%, after which increasing either the temporal window or graph depth yields almost no additional gain. These results indicate that attack context can be effectively reconstructed through a small set of temporally close and relationally connected alerts, rather than by simply extending an LLM’s sequential context window.

C. Observation 3: Contextual knowledge is abundant but unevenly informative

An isolated alert is often insufficient for reliable adjudication, yet simply providing more context does not necessarily resolve the ambiguity. As shown in Figure 6, a benign business request containing traversal-like sequences and a sensitive-looking path is conservatively classified by the LLM as an attack when evaluated without enterprise context. In practice, the available context may include many historical cases and surrounding events, but their diagnostic value is highly uneven: much of the information can be unrelated or redundant, while a small number of observations, such as a highly match-

```
[Alert Payload]
GET /xxxxx/portal/attachment/preview?module=workflow&
fileId=8f3a21c9&fileName=.%2F..%2F..%2FWEB-INF%2F
classes%2Fapplication.yml&download=false&tenantId=10001
HTTP/1.1

[LLM Adjudication Result]
The alert rule is Directory Traversal Attack, and the
request contains URL-encoded traversal sequences such as
'..%2F..%2F..%2F', attempting to access
'WEB-INF/classes/application.yml'. This file may contain
sensitive application configuration, credentials, or
internal service information. Even though the response
returns 403 Forbidden, the request itself indicates a
deliberate attempt to bypass directory restrictions. Under
a conservative security policy, such probing behavior
should be escalated as a potential attack attempt.
```

Fig. 6: A benign business request that triggers a directory-traversal alert and is misclassified by an LLM when evaluated without enterprise context.

ing benign pattern, preceding reconnaissance, or subsequent security-sensitive behavior, can substantially strengthen or invalidate an attack hypothesis. This observation motivates SOCRATES to selectively retrieve enterprise-specific false-positive knowledge and relevant behavioral evidence, rather than simply enlarging the LLM context.

IV. METHODOLOGY

As illustrated in Figure 7, in this section, we describe the system architecture and the design underlying its three modules in detail.

A. Module 1: Hierarchical Benign Fingerprint Filtering

To efficiently suppress recurring benign alerts before downstream analysis, SOCRATES first normalizes heterogeneous alerts and consolidates high-frequency behaviors, and then learns hierarchical benign fingerprints from analyst-confirmed benign examples. Only alerts that can be confidently matched to verified benign fingerprints are filtered, while uncertain cases are retained for subsequent graph-based analysis.

1) Alert Initialization and Time-Window Aggregation:

Alerts produced by heterogeneous detection systems differ in their field names, schemas, and available contextual information. Guided by the common alert abstractions defined in the Intrusion Detection Message Exchange Format (IDMEF) [7], SOCRATES maps each raw alert to a unified internal representation containing its alert source, normalized alert semantics, source and target entities, timestamp, protocol or service, and normalized attribute template. Because different alert types expose different security-relevant fields and payload structures, the attribute template is generated by an alert-type-specific normalization procedure rather than a single global template.

The resulting alert stream still contains high-frequency behaviors, such as scanning, probing, repeated authentication failures, and brute-force activity, in which a single behavior can produce many repetitive alert records. Before graph construction, SOCRATES aggregates configured high-frequency behaviors within a session-based time window using behavior-specific keys. For scanning and probing, the aggregation key consists of the source entity, normalized behavior category,

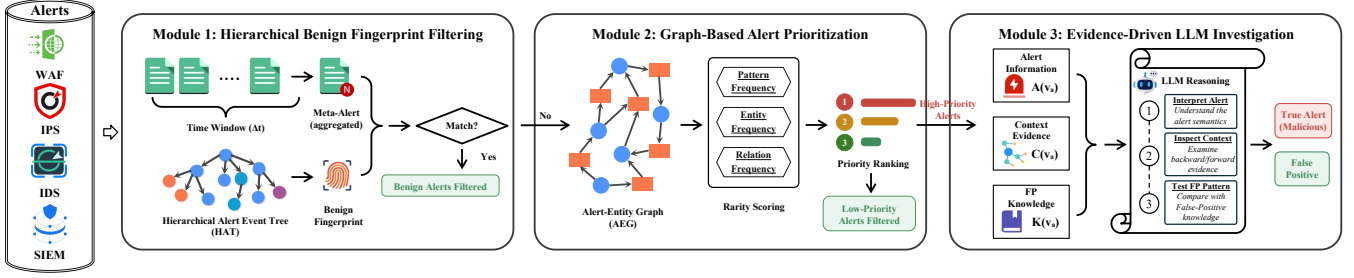


Fig. 7: System architecture of SOCRATES for automated and context-aware alert triage.

and protocol or service; the contacted targets and destination ports are allowed to vary and are accumulated as sets. For repeated authentication failures and brute-force activity, the key additionally requires a consistent target entity and authentication service, preventing attempts against different targets or services from being merged. An incoming alert is added to an active group only when its aggregation key is compatible with that group and the interval from the group’s latest alert does not exceed Δt ; otherwise, the active group is closed and a new group is initialized. Alerts outside the configured high-frequency behavior categories remain as individual alert units. When an active group is closed, it is represented as a meta-alert that records the identifiers of its constituent alerts, the first and last timestamps, the involved source and target entities, the normalized behavior template, and behavior-specific statistics, including the alert count and the numbers of distinct targets and destination ports. The meta-alert becomes the processing unit for subsequent graph-based analysis, while references to all original alerts are retained for contextual reconstruction and analyst review.

2) *Hierarchical Alert Event Tree Construction*: To recognize recurring benign behaviors beyond exact duplicates, SOCRATES organizes historical alerts into a **Hierarchical Alert event Tree (HAT)**, as illustrated in Figure 8. From the root to the leaf, the HAT contains six levels: alert source, alert semantics, source entity, target entity, service information, and attribute template. For endpoint alerts without an explicit source or target, a predefined null value preserves the same structure across alert sources. The alert source and semantics define coarse behavior branches, while the remaining levels describe the entities, services, and structural attributes that distinguish individual benign patterns. This organization allows different branches to retain different levels of specificity rather than applying one global normalization rule to all alerts.

The HAT is constructed exclusively from alerts confirmed to be benign in the training data. Instead of immediately replacing every variable value with a generic placeholder, SOCRATES retains both its concrete value and its type and structural shape. It first inserts each distinct benign alert as an exact root-to-leaf path and records its support and original alert references. Within each alert-source and alert-semantics branch, the system then consolidates paths that share the same security-relevant structure and differ only in instance-specific fields. Values that remain unchanged are preserved as exact

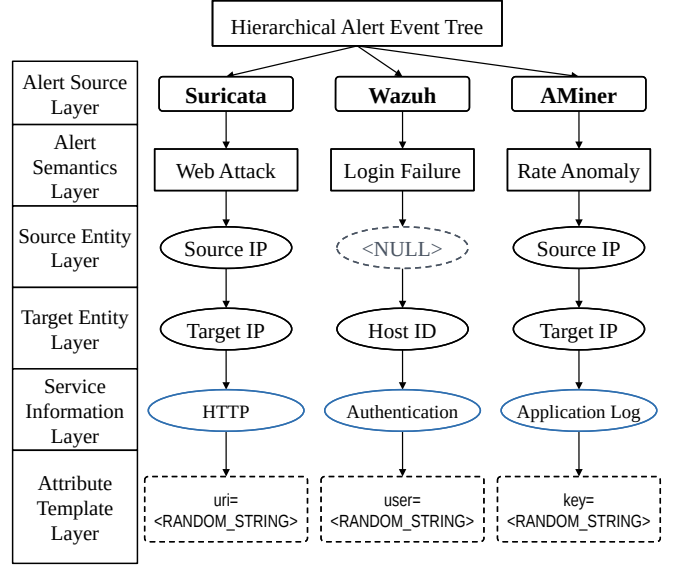


Fig. 8: Hierarchical alert event tree for organizing heterogeneous alerts.

constraints, whereas recurring variations with a consistent type and shape are represented using typed constraints, such as $\langle \text{TOKEN} \rangle$, $\langle \text{UID} \rangle$, or $\langle \text{HASH} \rangle$. Stable rule attributes, service characteristics, and payload tokens that define the alert behavior are not generalized.

A generalized branch is activated only when it is supported by multiple benign examples with consistent structural variations and does not overlap with another benign branch. Otherwise, the corresponding alerts remain as exact or inactive paths and are not eligible for automatic filtering. As a result, identifiers and other volatile values can be abstracted in branches where their variation is repeatedly observed, while entities, services, or payload structures remain concrete when they characterize a specific business behavior. Each active fingerprint retains its support, generalized fields, accepted value types and shapes, and links to the original alerts, enabling subsequent inspection and controlled updates after analyst verification.

3) *Benign Fingerprint Matching*: When a new alert arrives, SOCRATES maps it to the same six-level representation used during HAT construction and performs an exact-to-general matching procedure. For exact matching, the complete alert path is serialized into a canonical fingerprint, whose hash

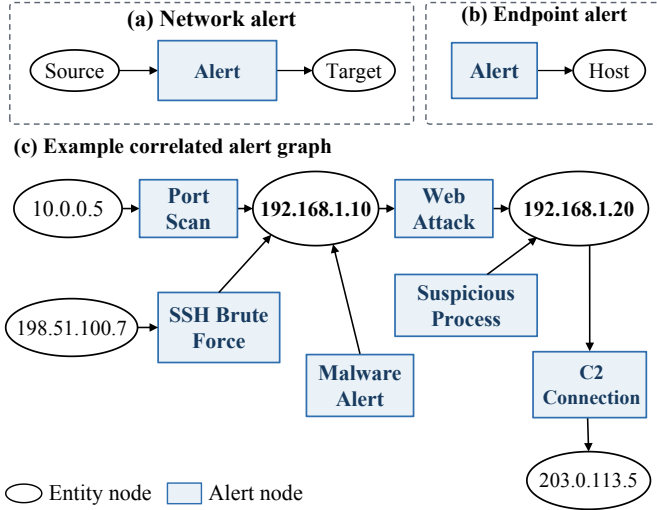


Fig. 9: Construction of the heterogeneous alert graph.

is used to efficiently retrieve an identical historical benign fingerprint. If exact matching fails, SOCRATES enters the corresponding Alert Source and Alert Semantics branch of the HAT and performs generalized matching from more specific patterns to stored generalized fingerprints. A generalized fingerprint matches only when the invariant fields and security-relevant structural features are consistent and the abstracted fields satisfy the learned type and shape constraints. An alert is filtered only when it can be uniquely matched to a valid benign fingerprint. If no valid fingerprint is found, the alert exhibits previously unseen structural variations or missing fields, or multiple fingerprints produce ambiguous matches, SOCRATES does not make a benign decision and conservatively forwards the alert for graph-based analysis.

B. Module 2: Graph-Based Alert Prioritization

After hierarchical benign fingerprint filtering, the remaining alert stream consists of individual alerts and meta-alerts that cannot be confidently identified as recurring benign behaviors. Directly forwarding all of these alerts to the LLM would still incur substantial computational overhead and expose the model to a large amount of low-value evidence. To further reduce the investigation space, SOCRATES performs graph-based alert prioritization. Specifically, it prioritizes each alert or meta-alert according to the rarity of its alert pattern, participating entities, and relations. Alerts with lower priority are conservatively filtered, while higher-priority candidates are retained for subsequent evidence-driven LLM investigation.

1) *Alert Graph Construction*: Prior studies and operational alert-correlation systems have shown that security events exhibit both temporal and relational locality: alerts occurring within a nearby time window and involving shared or closely related entities are more likely to belong to the same underlying activity [11], [19]. Motivated by this observation, we construct a directed heterogeneous **Alert-Entity Graph (AEG)** to capture the relational context that is absent from isolated alert records. As illustrated in Figure 9, the AEG

is denoted by $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from the individual alerts and meta-alerts retained by the first module. The node set $\mathcal{V} = \mathcal{V}_a \cup \mathcal{V}_e$ consists of two disjoint types of nodes: alert nodes $\mathcal{V}_a$ and entity nodes $\mathcal{V}_e$. Each alert node $v_a \in \mathcal{V}_a$ represents either an individual security event or an initialized meta-alert. An individual-alert node stores its alert type, timestamp, protocol, source and destination ports, service information, and other device-specific metadata. A meta-alert node additionally stores its aggregation interval, behavior statistics, involved entity sets, and references to its constituent alerts. Each entity node $v_e \in \mathcal{V}_e$ represents a network or endpoint entity involved in an alert, typically identified by an IP address or host identifier. The directed edge set $\mathcal{E}$ explicitly encodes the role of each entity in the corresponding event. An edge from an entity node to an alert node, $v_e \rightarrow v_a$, indicates that the entity acts as the source or initiator of the event, whereas an edge from an alert node to an entity node, $v_a \rightarrow v_e$, indicates that the entity is the destination, affected host, or target of the event. Accordingly, an individual network-side alert is represented as a directed triplet $\langle v_{src}, v_a, v_{dst} \rangle$, corresponding to the path $v_{src} \rightarrow v_a \rightarrow v_{dst}$. A meta-alert is connected to every entity involved in its constituent events using the same role-specific edge directions. For endpoint-side alerts that do not contain an explicit source entity, the event is represented as $v_a \rightarrow v_{host}$, where v_{host} denotes the affected endpoint.

2) *Graph-Based Anomaly Scoring*: After graph construction, the system assigns an anomaly score to each alert or meta-alert in the current-day alert graph. The scoring mechanism is motivated by operational observations and established prioritization practices, where repeatedly observed alerts generally carry lower investigation priority, while alerts exhibiting uncommon behavioral patterns, entities, or relationships warrant further analysis [14], [41], [45]. Because configured high-frequency activities have already been consolidated during alert initialization, each meta-alert is counted as a single occurrence during scoring, while its original alert count is retained only as an attribute.

For an alert node v_a , the system calculates three frequency values. First, the alert-pattern frequency $f_A(v_a)$ is the number of alert nodes that share the same normalized alert pattern, including alert semantics, protocol, service, and attribute template. Second, the entity frequency $f_E(v_a)$ measures how frequently the associated entities participate in the graph. For a network-side alert, it is calculated as the average participation frequency of its source and target entities; for an endpoint-side alert, only the frequency of the affected host is considered. Third, the relation frequency $f_R(v_a)$ counts how often the complete relation appears.

The three frequency values are independently normalized and converted into anomaly scores. For each component $x \in \{A, E, R\}$, the score is calculated as

$$S_x(v_a) = 1 - \frac{f_x(v_a) - f_x^{\min}}{f_x^{\max} - f_x^{\min}}, \quad (1)$$

where $f_x^{\min}$ and $f_x^{\max}$ denote the minimum and maximum frequencies observed in the current graph. Thus, a higher

occurrence frequency produces a lower anomaly score. The final score is obtained by combining the three components:

$$S(v_a) = \lambda_A S_A(v_a) + \lambda_E S_E(v_a) + \lambda_R S_R(v_a), \quad (2)$$

A low value of $S(v_a)$ indicates that the alert pattern, its associated entities, and their relation are all frequently observed in the current environment, and the alert can therefore be assigned a low investigation priority or filtered using a conservative threshold. Alerts with higher anomaly scores are retained and forwarded to the LLM-based adjudication stage for further contextual and semantic analysis.

C. Module 3: Evidence-Driven LLM Investigation

After hierarchical benign fingerprint filtering and graph-based alert prioritization, the remaining alerts are typically those that cannot be reliably resolved through deterministic matching or frequency-based statistics alone. To further investigate these ambiguous alerts, we introduce an evidence-driven LLM investigation module that integrates environment-specific false-positive knowledge with behavioral evidence extracted from the alert graph. As illustrated in Figure 10, the module consists of three major components. First, historical false-positive alerts are organized into a knowledge base and relevant benign patterns are retrieved for each alert under investigation. Second, bidirectional traversal over the alert graph reconstructs the preceding and subsequent behaviors surrounding the alert and converts them into structured contextual evidence. Finally, the alert information, retrieved false-positive knowledge, and contextual evidence are jointly provided to the LLM, which performs a structured evidence-guided analysis and produces the final adjudication result.

1) *False-Positive Knowledge Base Construction and Retrieval*: To provide the LLM with environment-specific knowledge of previously observed benign behaviors, we construct a false-positive knowledge base from alerts that have been confirmed as false positives. The knowledge base is built offline to capture recurring benign patterns while reducing the influence of alert-specific variations.

Specifically, we adopt a hierarchical clustering strategy to organize confirmed false-positive alerts. In the first stage, alerts are partitioned using relatively stable structured attributes, such as alert type, protocol, and service, so that alerts generated by fundamentally different security behaviors are separated into coarse-grained groups. Within each group, a second-stage clustering process further distinguishes alerts according to their normalized alert semantics and other security-relevant attributes. This hierarchical design prevents semantically unrelated alerts from being clustered solely because of superficial textual similarity, while allowing alerts with different dynamic fields but similar underlying behaviors to be grouped together. Each resulting cluster therefore represents a set of historical false-positive alerts exhibiting similar security semantics and operational characteristics.

For each false-positive cluster, we employ an LLM to extract a representative benign pattern and construct a knowledge

entry consisting of two components: an *index* and a *pattern*. Formally, a knowledge entry K_i is represented as

$$K_i = \langle I_i, P_i \rangle, \quad (3)$$

where I_i denotes the retrieval index and P_i denotes the corresponding benign pattern. The index I_i provides a concise semantic description of the alert behavior, including its major alert semantics, protocol or service characteristics, and other representative features. It is subsequently encoded into a vector representation for efficient similarity-based retrieval. In contrast, the pattern P_i contains a more detailed description of the common benign characteristics shared by alerts within the cluster and is provided to the LLM as environment-specific benign knowledge during alert investigation.

During online investigation, the false-positive knowledge base is queried through a two-stage retrieval procedure that follows the same hierarchical organization used during offline construction. Given an incoming alert a , the system first performs coarse-grained candidate filtering based on its structured attributes, including alert type, protocol, and service. Only knowledge entries belonging to compatible coarse-grained groups are retained as candidates. For fine-grained retrieval, we adopt a retrieval-augmented generation (RAG) scheme based on dense semantic representations. The normalized semantics and security-relevant attributes of the incoming alert are serialized into a textual query I_a using the same format as the indices generated during offline knowledge construction. Both I_a and the knowledge indices I_i are encoded by the same pretrained text embedding model, denoted by $\mathcal{E}(\cdot)$, producing dense vectors

$$\mathbf{q}_a = \mathcal{E}(I_a), \quad \mathbf{k}_i = \mathcal{E}(I_i). \quad (4)$$

The vectors of all offline knowledge indices are pre-computed and stored in a vector database with an approximate nearest-neighbor index. At runtime, the similarity between the query and each candidate index is measured using cosine similarity:

$$\text{sim}(I_a, I_i) = \frac{\mathbf{q}_a^\top \mathbf{k}_i}{\|\mathbf{q}_a\|_2 \|\mathbf{k}_i\|_2}. \quad (5)$$

The candidate entries are ranked according to their similarity scores, and the top- k entries exceeding a predefined similarity threshold τ_k are selected. The corresponding patterns P_i are then retrieved from the knowledge base and appended to the LLM input as environment-specific benign evidence. Importantly, the retrieved patterns are treated only as supporting knowledge rather than direct classification results. If no candidate satisfies the similarity threshold, the retrieved benign knowledge set is left empty, and the LLM proceeds with the subsequent investigation using only the available alert and contextual evidence.

2) *Bidirectional Alert Context Augmentation*: In addition to environment-specific benign knowledge, reliable alert investigation also requires sufficient behavioral context surrounding the alert under investigation. An isolated alert often provides only a partial view of the underlying activity and may be difficult to distinguish from benign behavior without considering what happened before and after it. Therefore, we

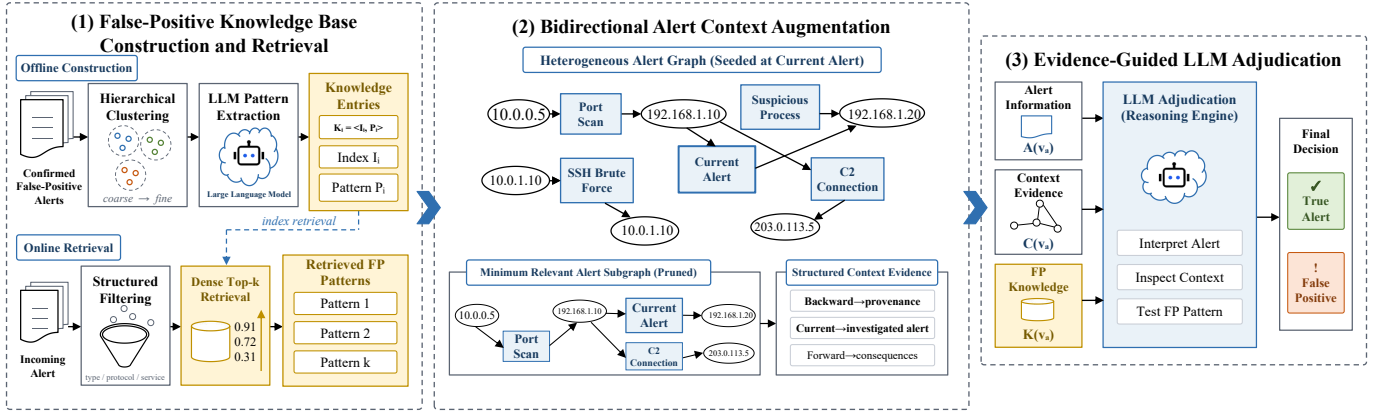


Fig. 10: Overview of the evidence-driven LLM investigation module.

further leverage the previously constructed AEG to retrieve contextually related events and reconstruct the local behavioral chain of each alert. Specifically, we first locate the alert under investigation in the AEG and use it to initialize a context-retrieval seed. Let v_a denote the current alert occurring at time t_a . For a network-side alert represented as $v_{src} \rightarrow v_a \rightarrow v_{dst}$, the seed set is defined as $\mathcal{S}_a = \{v_a, v_{src}, v_{dst}\}$, containing the alert itself and its source and target entities. For an endpoint-side alert represented as $v_a \rightarrow v_{host}$, the seed set becomes $\mathcal{S}_a = \{v_a, v_{host}\}$. These seed entities serve as anchors for retrieving temporally and relationally associated security events from the AEG.

Starting from $\mathcal{S}_a$, the system performs bidirectional context retrieval to reconstruct both the provenance and the possible consequences of the current alert. Backward retrieval searches the interval $[t_a - \Delta_b, t_a)$ for preceding alerts associated with the seed entities, such as earlier scans, service probes, authentication attempts, or other behaviors that may explain how the current event was reached. Forward retrieval searches $(t_a, t_a + \Delta_f]$ from the target or affected entity to identify subsequent behaviors that may reveal the outcome of the current event, such as successful authentication, suspicious command execution, privilege-related activities, or additional network connections. Candidate alerts are prioritized according to their temporal proximity and entity relationships with the current alert, with events involving the same source-target pair receiving higher priority than those sharing only one entity. The directional traversal therefore captures complementary evidence: backward paths describe the behavioral provenance of an alert, whereas forward paths reveal whether the alert is followed by security-relevant consequences. For online deployment, Δ_f is configured as a bounded observation window such that only evidence available before the final investigation decision is incorporated.

The retrieved nodes and edges are further pruned to construct a minimum relevant alert subgraph $\mathcal{G}_a \subseteq \mathcal{G}$ centered on v_a . The resulting subgraph is subsequently serialized into structured evidence before being provided to the LLM. Each evidence item is assigned a unique identifier and records its temporal relation to the current alert, alert semantics, involved

entities, and relevant behavioral statistics. Accordingly, the context of v_a can be represented as

$$C(v_a) = \{C_{backward}, C_{current}, C_{forward}\}, \quad (6)$$

where $C_{backward}$ summarizes preceding behaviors, $C_{current}$ describes the alert under investigation, and $C_{forward}$ summarizes subsequent behaviors. This compact representation preserves the most relevant behavioral chain while preventing redundant graph information from occupying the LLM context window, and provides directly referencable evidence for the subsequent alert adjudication process.

3) Evidence-Guided LLM Adjudication: After retrieving environment-specific benign knowledge and constructing the contextual evidence for the alert under investigation, we employ an LLM to perform the final alert adjudication. We organize the investigation as a structured multi-step reasoning process. For an alert v_a , the input to the LLM consists of three complementary sources of evidence: the original alert information $A(v_a)$, the bidirectional alert context $C(v_a)$ extracted from the alert graph, and the retrieved false-positive knowledge $K(v_a)$. The overall investigation input is therefore represented as

$$X(v_a) = \{A(v_a), C(v_a), K(v_a)\}. \quad (7)$$

If no relevant false-positive pattern is retrieved, $K(v_a)$ is set to empty and the investigation proceeds using only the alert itself and its contextual evidence.

The LLM is guided to analyze the collected evidence in a fixed sequence. First, it interprets the current alert and identifies the security-relevant behavior indicated by its semantics, entities, service information, and other attributes. Second, it examines the retrieved backward and forward contextual evidence to determine whether the current alert is part of a coherent attack progression. For example, preceding scanning or authentication attempts may strengthen the interpretation of a subsequent exploitation alert, while successful login, suspicious command execution, or outbound connections occurring afterward may provide evidence of a successful compromise. Third, the LLM examines the retrieved false-positive patterns and determines whether their described benign characteristics are applicable to the current alert. Importantly, a retrieved

benign pattern is not accepted solely because of semantic similarity. The model is additionally required to search the alert and contextual evidence for conflicting observations that invalidate the benign explanation, such as a behavioral transition or a subsequent security-sensitive event that is inconsistent with the historical false-positive pattern. Based on the above evidence, the LLM produces a final adjudication result. The decision is restricted to two outcomes: *True Alert* or *False Positive*.

V. EVALUATION

We conduct a comprehensive evaluation of SOCRATES in automated alert triage. Specifically, we aim to answer the following research questions:

- **RQ1: Overall Effectiveness:** What is the overall effectiveness of SOCRATES?
- **RQ2: Stage-wise Effectiveness:** How effective is each stage of SOCRATES in progressively reducing alerts and preserving genuine attacks?
- **RQ3: Comparison with Existing Approaches:** How does SOCRATES compare with representative baseline methods for alert triage?
- **RQ4: Ablation Study:** How do the key components of SOCRATES individually contribute to its overall alert triage performance?
- **RQ5: Industrial Deployment:** How effective and efficient is SOCRATES when deployed in a real-world enterprise environment?

A. Evaluation Setup

1) *Dataset Preparation:* We primarily evaluate SOCRATES on the publicly available AIT Alert Data Set (AIT-ADS) [21], [22], a synthetic alert dataset generated from the AIT-LDSv2 testbed for multi-step attack analysis. AIT-ADS consists of eight independently instantiated enterprise network scenarios, each containing simulated normal user activities and multi-stage attacks covering reconnaissance, service enumeration, web exploitation, password cracking, remote command execution, privilege escalation, service disruption, and DNS-based data exfiltration. Alerts are generated by three heterogeneous detection systems (Wazuh, Suricata, and AMiner) and the dataset provides 79 annotated attack-phase intervals. As summarized in Table I, the eight scenarios contain 2,655,821 alerts in total, with substantial variation in alert volume across independently generated environments.

To complement the public benchmark, we further evaluate SOCRATES on a private Industry Arena Dataset collected from a production enterprise SOC. Details are provided in § V-F.

2) *Evaluation Metrics:* We evaluate SOCRATES using four complementary metrics:

- **True Positive Rate (TPR):** $TPR = TP / (TP + FN)$, measuring the proportion of true attacks that are correctly identified.
- **False Positive Rate (FPR):** $FPR = FP / (FP + TN)$, indicating the fraction of benign alerts incorrectly classified as attacks.

TABLE I: Scenario-level statistics of the AIT-ADS dataset.

Scenario	# Alerts	Size (GB)	Time Range
fox	473,104	0.5003	Jan. 15–19, 2022
harrison	593,948	0.6416	Feb. 4–8, 2022
russellmitchell	45,544	0.0487	Jan. 21–24, 2022
santos	130,779	0.1491	Jan. 14–17, 2022
shaw	70,782	0.0767	Jan. 25–30, 2022
wardbeck	91,257	0.0991	Jan. 19–23, 2022
wheeler	616,161	0.6609	Jan. 26–30, 2022
wilson	634,246	0.6813	Feb. 3–8, 2022
Total	2,655,821	2.8577	/

- **F1-score:** $F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$, providing a balanced measure of triage accuracy.
- **Alert Reduction Ratio (ARR):** $ARR = 1 - N_{\text{triaged}} / N_{\text{total}}$, representing the proportion of alerts filtered out.

3) *Experimental Settings: Data Splits.* We construct the training and test sets independently for each AIT-ADS scenario to prevent information leakage across enterprise environments. Specifically, we sort alerts by timestamp and use the earliest 20% of the benign alerts outside the annotated attack intervals to initialize the HAT. To construct the false-positive knowledge base, we sample 20% of the false-positive alerts within the annotated attack intervals. The two training subsets are disjoint, and all remaining alerts, including all annotated attack alerts, are used exclusively for testing. We use a random seed for stratified sampling and knowledge construction.

Model Configuration. We use DeepSeek-V4-Flash as the reasoning model for both false-positive pattern extraction and final alert adjudication, and Qwen3-Embedding-0.6B as the text embedding model for knowledge retrieval. Neither model is fine-tuned on the evaluation datasets. Qwen3-Embedding-0.6B produces 1,024-dimensional vectors and is deployed locally using Ollama. For each alert, SOCRATES first performs an exact semantic-type gate and then retrieves at most five false-positive patterns whose cosine similarity is no lower than 0.7. DeepSeek-V4-Flash is queried with temperature 0 and a maximum output length of 800 tokens. We use the same prompts and decoding parameters for all datasets and alert sources.

B. RQ1: Overall Effectiveness

We first evaluate the overall effectiveness of SOCRATES. The results are summarized in Table II and Table III.

Progressive Alert Compression. As shown in Table II, SOCRATES progressively reduces the alert stream through its three-stage coarse-to-fine pipeline. Module 1 reduces 2,477,576 raw alerts to 250,639, achieving a stage-wise ARR of 89.8837%. Module 2 further reduces the remaining alert set from 250,639 to 42,355, corresponding to a stage-wise ARR of 83.1012% and increasing the cumulative ARR to 98.2905%. Module 3 performs the final adjudication over the remaining 42,355 candidates and reduces them to only

TABLE II: Stage-wise alert compression performance of SOCRATES.

Stage	# Input Alerts	# Output Alerts	Stage ARR $\uparrow$	Total ARR $\uparrow$
Module 1	2,477,576	250,639	89.8837%	89.8837%
Module 2	250,639	42,355	83.1012%	98.2905%
Module 3	42,355	709	98.3261%	99.9714%
Total	2,477,576	709	–	99.9714%

TABLE III: Stage-wise and end-to-end triage performance of SOCRATES.

Stage	# Input Alerts	# Output Alerts	TPR $\uparrow$	FPR $\downarrow$	F1 $\uparrow$
Module 1	2,477,576	250,639	99.9998%	48.1668%	90.8183%
Module 2	250,639	42,355	99.9999%	35.1803%	96.5654%
Module 3	42,355	709	99.9973%	3.7968%	99.8638%
Total	2,477,576	709	99.9970%	0.6434%	99.8637%

709 alerts, achieving a stage-wise ARR of 98.3261%. Overall, SOCRATES reduces the original 2,477,576 alerts to 709 analyst-facing alerts, corresponding to an end-to-end ARR of 99.9714%. These results demonstrate that the coarse-to-fine design progressively contracts the investigation space at each stage, transforming a million-scale raw alert stream into a compact set of alerts requiring analyst attention.

Beyond the overall alert compression capability, we further evaluate the triage accuracy of each individual module and the complete SOCRATES pipeline. For each module, we report the input and output alert volumes together with the TPR, FPR, and F1-score computed over the alerts processed at that stage. The results are summarized in Table III.

High-Accuracy Triage. As shown in Table III, SOCRATES progressively improves its ability to distinguish genuine attacks from benign alerts while maintaining near-perfect attack retention throughout the pipeline. Module 1 achieves a TPR of 99.9998%, but its FPR remains relatively high at 48.1668%, reflecting its conservative design that preserves uncertain alerts for subsequent analysis. Module 2 further improves discrimination, reducing the FPR to 35.1803% and increasing the F1-score from 90.8183% to 96.5654%, while maintaining a TPR of 99.9999%. Module 3 operates on the substantially harder residual alert set and further reduces the stage-level FPR to 3.7968%, while achieving a TPR of 99.9973% and an F1-score of 99.8638%. When evaluated end-to-end over the original 2,477,576 alerts, the complete SOCRATES pipeline achieves a TPR of 99.9970%, an FPR of only 0.6434%, and an F1-score of 99.8637%. These results demonstrate that the three-stage design progressively eliminates benign ambiguity while preserving nearly all attack alerts, enabling highly accurate triage despite the aggressive reduction of the original alert stream.

C. RQ2: Stage-wise Effectiveness

To better understand how each stage contributes to the overall effectiveness of SOCRATES, we conduct a stage-wise

analysis of the three modules in the pipeline. We examine how alerts are progressively processed at each stage, which can provide a more fine-grained view of the role and effectiveness of each module in the coarse-to-fine triage process.

Complementary Early-Stage Reduction. Figure 11 shows how Module 1 processes alerts across the eight AIT-ADS scenarios. For each scenario, the first bar partitions the original alerts into HAT-filtered alerts, alerts routed through aggregation, and non-aggregated alerts directly forwarded to Module 2, while the second bar shows the actual volume remaining after aggregation. Aggregation dominates in fox, harrison, wheeler, and wilson, where 74.7%–91.9% of alerts are routed through this branch and compressed to only 0.3%–0.4% of the original volume. In contrast, HAT filtering contributes more substantially in russellmitchell, santos, shaw, and wardbeck, directly removing 36.4%–51.9% of alerts. Overall, only 2.4%–31.4% of the original alerts are forwarded to Module 2 across the eight scenarios. These results demonstrate that aggregation and HAT filtering provide complementary reduction capabilities: the former compresses bursty repetitive activities, whereas the latter removes recurring benign patterns.

Effective Graph-Based Prioritization. Figure 12 shows the anomaly-score distributions of attack and non-attack alerts produced by Module 2, where the vertical axis is shown on a logarithmic scale and the prioritization threshold is set to 0.7. Attack alerts are predominantly concentrated in the high-score region, whereas non-attack alerts are distributed more broadly and constitute the majority of alerts below the threshold. This separation enables Module 2 to eliminate a large volume of low-priority alerts while retaining suspicious candidates for further investigation. Meanwhile, a non-negligible number of non-attack alerts also receive scores above 0.7, indicating that statistical rarity alone cannot reliably distinguish previously unseen benign behaviors from genuine attacks. Therefore, the anomaly score is used as a prioritization signal rather than a final security verdict: low-score alerts are conservatively filtered, while high-score candidates are forwarded to Module 3 for evidence-driven contextual adjudication.

Evidence-Grounded Alert Adjudication. Figure 13 illustrates how Module 3 resolves an ambiguous alert by jointly reasoning over the original alert, its graph-based behavioral context, and retrieved false-positive knowledge. The alert itself reports a Suricata TLS invalid-handshake message, which exposes suspicious protocol semantics but provides no direct evidence of exploitation or compromise. The retrieved context contains temporally related alerts connected through the alert graph, yet these neighboring events do not form a coherent attack progression. Meanwhile, the false-positive knowledge base retrieves a matching historical pattern indicating that TLS compatibility quirks can be mistakenly flagged by Suricata’s strict protocol decoder. By combining these complementary observations, the LLM determines that the suspicious alert semantics are sufficiently explained by the known benign pattern and are not contradicted by the surrounding behavior, and therefore correctly adjudicates the alert as a false positive.

Due to space limitations, detailed stage-wise results are

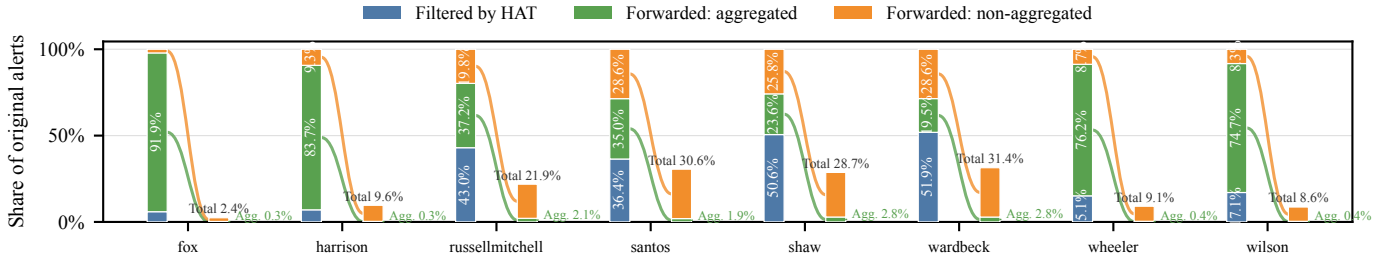


Fig. 11: Scenario-level alert routing and compression in Module 1.

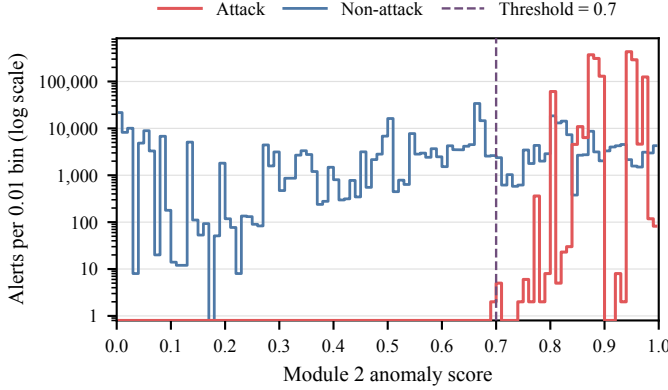


Fig. 12: Distribution of Module 2 anomaly scores for attack and non-attack alerts.

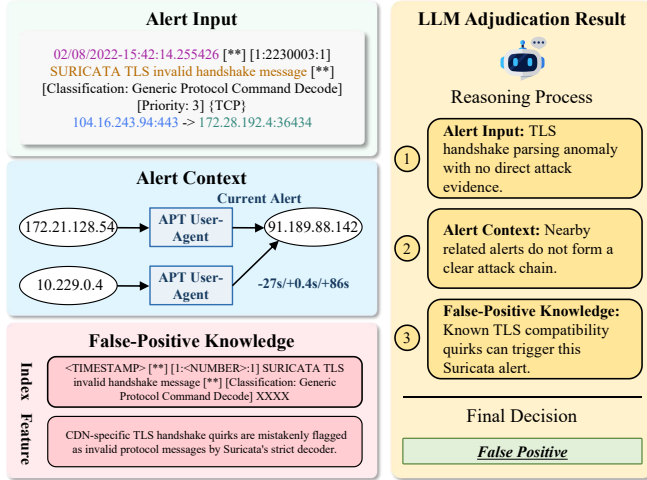


Fig. 13: Case study of evidence-grounded alert adjudication in Module 3.

provided in § VI-D.

D. RQ3: Comparison with Existing Approaches

We compare SOCRATES with representative traditional baselines and prior alert-triage approaches evaluated on AIT-ADS. The traditional baselines include Attack-Window Filtering, which retains alerts occurring within known attack intervals, and Historical Category Rate, which predicts alert relevance according to the historical malicious rate of each alert category. We further compare against DeepCase [39],

TABLE IV: Comparison with representative alert-triage approaches on AIT-ADS.

Approach	TPR↑	FPR↓	F1↑	ARR↑
<i>Traditional Baselines</i>				
Attack-Window Filtering	100.00%	2.1457%	99.4432%	33.56%
Historical Category Rate*	87.55%	—	91.90%	95.45%
<i>Prior Alert-Triage Methods</i>				
DeepCase† [39]	66.97%	—	79.24%	89.74%
AACT [38]	90.97%	—	94.95%	95.45%
PACT§ [26]	76.34±31.44%	0.0150±0.0088%	84.13±20.45%	—
NLLog¶ [25]	89.80%	0.90%	93.90%	—
GARNET‡ [12]	—	0.1067%	67.6733%	79.15%
<i>Ours</i>				
SOCRATES	99.9970%	0.6434%	99.8637%	99.9714%

* Historical Category Rate uses the historical malicious rate of each alert category as the prediction score; the result is reported by AACT on AIT-ADS.

† DeepCase results are reproduced and reported by AACT under its AIT-ADS evaluation protocol.

§ PACT evaluates a preprocessed event stream under a different chronological evaluation protocol; results are included for contextual comparison.

¶ NLLog reports session-level anomaly-classification results rather than end-to-end alert-level triage results.

‡ GARNET uses only AMiner alerts, with harrison and santos for training and the remaining six AIT-ADS scenarios for evaluation. Its FPR and F1 are macro-averages computed from the six scenario-level results reported by the authors; GARNET reports ECR rather than ARR.

AACT [38], PACT [26], NLLog [25], and GARNET [12]. Table IV summarizes the reported results.

As shown in Table IV, simple temporal or historical heuristics can already achieve competitive performance on AIT-ADS, but exhibit clear limitations in balancing attack preservation and alert reduction. Attack-Window Filtering achieves a TPR of 100% and an F1-score of 99.4432% because it directly relies on the ground-truth attack intervals. However, it reduces only 33.56% of the original alerts and still produces an FPR of 2.1457%, since benign alerts occurring within attack windows are conservatively retained. Historical Category Rate achieves a substantially higher ARR of 95.45%, but its TPR decreases to 87.55%, showing that category-level historical statistics alone cannot reliably preserve attack alerts.

Compared with prior alert-triage approaches, SOCRATES achieves a stronger overall balance among attack preservation, triage accuracy, and alert reduction. DeepCase and AACT achieve F1-scores of 79.24% and 94.95%, respectively, while AACT retains 90.97% of attack alerts with an ARR of 95.45%. In contrast, SOCRATES achieves a TPR of 99.9970%, an F1-score of 99.8637%, and an ARR of 99.9714% over the complete AIT-ADS corpus, demonstrating that it can aggressively reduce the analyst-facing alert volume while preserving nearly all attack alerts. Although PACT and GARNET report lower

TABLE V: Ablation study of the denoising components in Modules 1 and 2.

Variant	# Input Alerts	# Output Alerts	ARR↑	TPR↑	FPR↓	F1↑
Full M1 + M2		42,355	98.2905%	99.9997%	16.9452%	96.5654%
w/o M1 Aggregation		174,573	92.9539%	0.4874%	22.6701%	0.8862%
w/o M1 HAT	2,477,576	142,156	94.2623%	100.0000%	46.7790%	91.0593%
w/o Module 1		341,678	86.2092%	0.4878%	45.4811%	0.8158%
w/o Module 2		250,639	89.8837%	99.9998%	48.1668%	90.8183%

FPRs under their respective settings, PACT is evaluated on a preprocessed event stream under a different chronological protocol, while GARNET considers only AMiner alerts from six evaluation scenarios; NLog further operates at the session level rather than the alert level. These protocol differences limit direct numerical comparison with our full-corpus alert-level evaluation. Overall, these results show that SOCRATES provides a more favorable trade-off between attack preservation and alert reduction under a full-corpus, alert-level evaluation setting.

E. RQ4: Ablation Study

The three modules of SOCRATES serve two distinct purposes: Modules 1 and 2 progressively reduce the alert stream, whereas Module 3 focuses on semantic adjudication of the remaining ambiguous alerts. We therefore conduct two complementary ablation studies rather than removing all modules under a single end-to-end setting. First, we evaluate the contribution of the denoising mechanisms in Modules 1 and 2 on the complete AIT-ADS alert stream by disabling aggregation, HAT filtering, and graph-based prioritization individually or jointly. Second, we investigate which information sources and reasoning components affect the LLM’s final decisions by removing alert context, false-positive knowledge, or CoT-based reasoning from Module 3. Since the effects of different LLM configurations would be diluted by the millions of alerts already removed upstream, we construct a balanced evaluation subset from the output of Module 2 by randomly sampling 300 attack alerts and 300 false-positive alerts for the second experiment.

Contribution of Denoising Components. As shown in Table V, the full combination of Modules 1 and 2 achieves an ARR of 98.2905% while retaining 99.9997% of attack alerts. Removing aggregation causes the largest degradation, reducing the TPR to only 0.4874%. This is because Module 2 relies on the frequencies of alert patterns, entities, and relations for anomaly scoring; without aggregation, bursty behaviors generate many repetitive alerts and distort these frequency statistics. In the full pipeline, such alerts are first consolidated into meta-alerts and counted as single occurrences during scoring. Removing HAT, in contrast, preserves a TPR of 100% but increases the FPR from 16.9452% to 46.7790%, showing its importance in suppressing recurring benign behaviors. Removing Module 2 also leaves a high FPR of 48.1668%. Overall, aggregation, HAT filtering, and graph-based prioritization provide complementary denoising effects.

Contribution of Evidence and Reasoning. As shown in Table VI, the full Module 3 configuration achieves the highest

TABLE VI: Ablation study of evidence and reasoning components in Module 3 on the balanced evaluation subset.

Variant	TPR↑	FPR↓	F1↑
Full Module 3	63.0000%	14.3333%	71.0526%
w/o Alert Context	35.6667%	8.3333%	49.5370%
w/o FP Knowledge	60.6667%	21.6667%	66.5448%
w/o CoT	60.3333%	20.6667%	66.6667%

F1-score of 71.0526% and a TPR of 63.0000%, indicating the best overall balance between attack preservation and false-positive suppression. Removing alert context causes the largest degradation, reducing the TPR to 35.6667% and the F1-score to 49.5370%, showing that surrounding behavioral evidence is critical for recognizing genuine attacks, even though the resulting FPR becomes lower. Removing false-positive knowledge or CoT-based reasoning produces smaller TPR drops but substantially increases the FPR to 21.6667% and 20.6667%, respectively. These results indicate that behavioral context primarily supports attack recognition, while false-positive knowledge and structured reasoning help the LLM avoid incorrectly escalating benign alerts.

F. RQ5: Industrial Deployment

By the time of this writing, SOCRATES has been deployed within the SOC of a large-scale enterprise for over one year, monitoring a network infrastructure consisting of more than 10,000 endpoint devices. During this extended deployment, the system has demonstrated exceptional stability under high-throughput conditions, processing an average of over one million raw alerts daily. By effectively filtering benign anomalies and aggregating redundant events, SOCRATES achieves a remarkable alert noise reduction rate of 99.6%, which has translated into a reduction in manual investigation effort by more than 80%. To further demonstrate the operational effectiveness of SOCRATES, we construct an *Industry Arena Dataset* from alerts collected during the production deployment and use it for additional evaluation. Due to space limitations, detailed descriptions of the dataset and experiment are provided in § VI-E.

VI. CONCLUSION

In this paper, we present SOCRATES, an AI-powered SOC agent for scalable and evidence-grounded network alert triage. Motivated by the repetitive nature of alert streams, the non-sequential structure of attack context, and the uneven diagnostic value of contextual evidence, SOCRATES adopts a coarse-to-fine architecture that progressively reduces redundant alerts, prioritizes anomalous candidates, and applies evidence-driven LLM reasoning only to ambiguous cases. On AIT-ADS, SOCRATES reduces 2,477,576 test alerts to 709 analyst-facing alerts, achieving a 99.9714% ARR, 99.9970% TPR, 0.6434% FPR, and 99.8637% F1-score. Stage-wise and ablation studies further validate the complementary roles of the proposed components, while long-term deployment and evaluation in a production enterprise SOC demonstrate the practicality of SOCRATES under real-world workloads.

A. Ethical Considerations

Automated alert triage introduces operational risks because an incorrect benign decision may reduce visibility into a genuine attack, while excessive escalation can recreate the alert-fatigue problem that the system aims to alleviate. SOCRATES therefore adopts a conservative coarse-to-fine design: HAT automatically filters an alert only when it uniquely matches a valid benign fingerprint, uncertain alerts are retained for subsequent analysis, and the final LLM adjudication is grounded in explicitly retrieved behavioral and historical evidence. The system also preserves references to original alerts and traceable evidence throughout aggregation and investigation, enabling analysts to inspect the basis of automated decisions.

Deployment also requires careful handling of enterprise security telemetry, which may contain sensitive network identifiers, payloads, and behavioral information. Access to such data and the derived false-positive knowledge should therefore be restricted to authorized security personnel, and enterprise-specific knowledge should not be reused outside its intended environment without appropriate authorization. Because analyst-confirmed false positives are reused to construct HAT fingerprints and the knowledge base, updates to these components should remain controlled and auditable to reduce the risk of erroneous or malicious feedback contaminating future decisions. Finally, SOCRATES is intended to support alert triage and investigation rather than autonomously execute high-impact response actions; containment, remediation, and other consequential security operations should remain subject to human oversight.

B. Discussion

Architectural Implications. Our results suggest that the practical value of LLMs in SOC automation depends not only on their reasoning capability, but also on how they are positioned within the alert-processing pipeline. Directly applying an LLM to raw alert streams wastes inference capacity on repetitive evidence and exposes the model to substantial benign noise. In contrast, SOCRATES separates large-scale denoising from semantic investigation: deterministic and statistical mechanisms first reduce and prioritize the alert stream, while the LLM is reserved for the small set of ambiguous cases that require contextual reasoning. This design also highlights the importance of enterprise-specific knowledge and structured behavioral context, which provide the evidence needed to interpret suspicious alert semantics beyond isolated records. More broadly, these findings support a hybrid SOC architecture in which lightweight mechanisms handle volume and repetition, while LLMs complement human analysts by performing evidence-grounded investigation on high-value cases.

Limitations. Despite the strong empirical results, SOCRATES has several limitations. First, both HAT filtering and the false-positive knowledge base rely on analyst-confirmed benign examples. Their effectiveness may therefore be limited

in newly deployed environments with insufficient historical knowledge, and incorrect analyst labels may propagate into subsequent filtering or retrieval. Second, the early-stage reduction mechanisms depend on environment-specific alert distributions and configuration parameters. In particular, Module 2 derives anomaly scores from the frequencies of alert patterns, entities, and relations, making its prioritization sensitive to substantial workload or behavioral shifts if high-frequency activities are not properly aggregated. Third, the quality of LLM adjudication is bounded by the evidence available to the system. Missing telemetry, incomplete graph connections, or evidence outside the configured backward and forward observation windows may prevent reconstruction of the complete attack progression.

C. Open Science

Artifacts related to this paper are available at <https://anonymous.4open.science/r/SOCRATES-DAC4>.

D. Detailed Stage-wise Evaluation Results

We first report the detailed per-scenario performance of Module 1 to further illustrate how hierarchical benign filtering and high-frequency behavior aggregation jointly reduce the alert workload across different AIT-ADS environments.

Table VII summarizes the processing results for all eight scenarios. For each scenario, we report the raw input volume, the number of alerts directly removed by HAT, the number of high-frequency alert members and their corresponding aggregated meta-alerts, and the non-aggregated alerts forwarded unchanged. The Module 1 output consists of the generated meta-alerts and non-aggregated forwarded alerts. We additionally report the resulting alert reduction ratio (ARR) and attack retention rate (TPR). As shown in **Table VII**, the relative contribution of the two reduction mechanisms varies substantially across scenarios. Aggregation dominates in *fox*, *harrison*, *wheeler*, and *wilson*, where hundreds of thousands of high-frequency alerts are compressed into only a few thousand meta-alerts. In contrast, HAT filtering contributes more strongly in *russellmitchell*, *santos*, *shaw*, and *wardbeck* by directly removing recurring benign alerts. Overall, Module 1 reduces 2,477,576 raw alerts to 250,639 processing units, achieving an ARR of 89.8837% while retaining 99.9998% of attack alerts. Specifically, 379,701 alerts are filtered by HAT, while 1,861,415 high-frequency alerts are consolidated into only 14,179 meta-alerts, demonstrating the complementary effects of benign fingerprint filtering and behavior aggregation.

Following the substantial workload reduction achieved by Module 1, we further examine how Module 2 prioritizes the remaining alert objects and reduces the candidate set for subsequent LLM investigation. **Table VIII** reports the per-scenario results of graph-based alert prioritization. For each scenario, we report the number of Module 2 input objects, the objects assigned below the prioritization threshold as non-candidates, and the remaining candidate objects forwarded to Module 3. M2-ARR measures the reduction at the operational-object level, while TPR, FPR, and F1 are evaluated at the

TABLE VII: Per-scenario workload reduction and attack retention of Module 1 on AIT-ADS.

Scenario	# Raw Input	# HAT-Filtered Members	Aggregated Members → Meta-Alerts	# Non-Aggregated Forwarded	# Module 1 Output	ARR↑	TPR↑
fox	462,814	27,435	425,520 → 1,288	9,859	11,147	97.5915%	99.9995%
harrison	561,457	39,412	469,788 → 1,858	52,257	54,115	90.3617%	100.0000%
russellmitchell	38,839	16,697	14,464 → 814	7,678	8,492	78.1354%	100.0000%
santos	107,224	39,072	37,476 → 2,083	30,676	32,759	69.4481%	100.0000%
shaw	58,013	29,353	13,673 → 1,653	14,987	16,640	71.3168%	100.0000%
wardbeck	74,414	38,616	14,538 → 2,105	21,260	23,365	68.6013%	99.9836%
wheeler	579,396	87,590	441,339 → 2,223	50,467	52,690	90.9060%	100.0000%
wilson	595,419	101,526	444,617 → 2,155	49,276	51,431	91.3622%	100.0000%
Total	2,477,576	379,701	1,861,415 → 14,179	236,460	250,639	89.8837%	99.9998%

Note: All results are measured on the common chronological TEST split. HAT-filtered and aggregated counts are expanded to their original raw members, whereas meta-alerts and Module 1 outputs are counted as operational alert objects. The three member-level routing categories are mutually exclusive and sum to the raw input. ARR is defined as $1 - N_{M1\text{ output}}/N_{\text{raw input}}$, and TPR is computed at the original-member level.

original-alert level by propagating each meta-alert decision to its constituent alerts. As shown in Table VIII, Module 2 further reduces 250,639 input objects to 42,355 candidates, corresponding to an M2-ARR of 83.1012%, while retaining 99.9997% of attack alerts. The reduction remains substantial across all eight scenarios, with M2-ARR ranging from 65.0600% to 91.3001%. In most scenarios, the prioritization removes the majority of low-anomaly objects while preserving nearly all attacks. However, *harrison* and *santos* exhibit relatively high FPRs of 40.4167% and 32.0886%, respectively, indicating that statistical rarity alone cannot reliably distinguish uncommon benign activities from genuine attacks. This result further motivates forwarding the remaining high-priority candidates to Module 3 for evidence-driven adjudication.

Finally, we examine the detailed performance of Module 3 on the high-priority candidates retained by Module 2, focusing on its ability to further suppress false positives while preserving genuine attacks.

As shown in Table IX, Module 3 reduces 42,355 candidate objects to only 709 retained objects, achieving an ARR of 98.3261% while maintaining a TPR of 99.9973% and an F1-score of 99.8638%. Across all eight scenarios, the TPR remains above 99.85%, indicating that the evidence-driven investigation preserves nearly all genuine attacks even on the difficult residual alert set. Meanwhile, Module 3 substantially suppresses the false positives remaining after graph-based prioritization, yielding an overall FPR of 3.7968%. Although the scenario-level FPR varies with workload characteristics, the consistently high attack retention and substantial candidate reduction demonstrate that behavioral context and enterprise-specific false-positive knowledge effectively resolve ambiguities that cannot be handled by statistical prioritization alone.

E. Industry Arena Dataset and Experimental Evaluation

To further assess the effectiveness of SOCRATES in a real-world SOC environment, we introduce a private Industry Arena Dataset collected from a production enterprise over 21 consecutive days, from January 5 to January 25, 2026. The dataset contains security alerts from two operational detection categories: network intrusion detection and Web intrusion

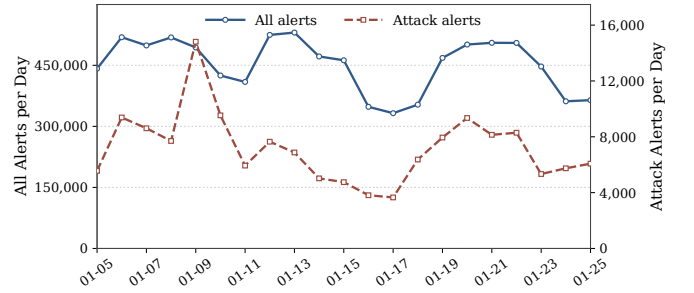


Fig. 14: Daily volumes of all alerts and confirmed attack alerts in the Industry Arena Dataset over 21 consecutive days.

detection. Each alert includes a timestamp, detector and rule identifiers, source and destination entities, ports and protocols, severity and confidence information, attack semantics, and packet- or application-level context when available. In total, the dataset contains 5,947,990 alerts. As shown in Figure 14, the operational environment consistently generates several hundred thousand alerts per day, whereas confirmed attack alerts constitute a much smaller and temporally varying subset. These alerts cover diverse network attack types, including SQL injection, directory traversal, and other Web- and network-based attacks.

We use the first 14 days of the Industry Arena Dataset as a warm-up period, during which alerts labeled by professional SOC operators are used to construct the HAT and the enterprise-specific false-positive knowledge base. The remaining seven days are used exclusively for testing, ensuring a chronological separation between knowledge construction and evaluation. All other system configurations and model settings follow those used in the main evaluation.

As shown in Table X, SOCRATES consistently achieves substantial alert reduction under the dynamically varying workload of the production SOC. During the seven-day test period, the system processes 1,938,725 raw alerts, with daily volumes ranging from 221,778 to 318,788. Module 1 reduces the stream to 107,401 alert objects, corresponding to an overall ARR of 94.46%, while Module 2 further reduces it to only 3,378 candidates, yielding a cumulative ARR of 99.83%. This

TABLE VIII: Per-scenario workload reduction and attack retention of Module 2 on AIT-ADS.

Scenario	# Module 2 Input Objects	# Non-Candidate Objects	# Candidate Objects	M2-ARR↑	TPR↑	FPR↓	F1↑
fox	11,147	9,877	1,270	88.6068%	99.9990%	6.6550%	99.6607%
harrison	54,115	38,579	15,536	71.2908%	100.0000%	40.4167%	93.9390%
russellmitchell	8,492	7,675	817	90.3792%	100.0000%	8.4753%	90.2673%
santos	32,759	21,313	11,446	65.0600%	100.0000%	32.0886%	42.0100%
shaw	16,640	15,092	1,548	90.6971%	100.0000%	6.1961%	79.0170%
wardbeck	23,365	20,797	2,568	89.0092%	99.9836%	7.8077%	69.5612%
wheeler	52,690	48,106	4,584	91.3001%	100.0000%	8.8137%	98.4993%
wilson	51,431	46,845	4,586	91.0832%	100.0000%	7.1554%	98.6947%
Total	250,639	208,284	42,355	83.1012%	99.9997%	16.9452%	96.5654%

Note: Module 2 input and output volumes are counted as operational alert objects. Non-candidate objects are Module 1 outputs whose graph anomaly score is below the candidate threshold. M2-ARR is defined as $1 - N_{\text{candidate}}/N_{\text{M2 input}}$. TPR, FPR, and F1 are measured at the original-member level by propagating each meta-alert decision to all of its constituent alerts.

TABLE IX: Per-scenario investigation performance and workload reduction of Module 3 on AIT-ADS.

Scenario	# Input Candidate Objects	# Retained Objects	TPR↑	FPR↓	F1↑	ARR↑
fox	1,270	54	99.9993%	20.9457%	99.9285%	95.7480%
harrison	15,536	126	99.9986%	1.4385%	99.9066%	99.1890%
russellmitchell	817	50	99.9361%	5.5861%	99.3694%	93.8800%
santos	11,446	133	99.9373%	2.5792%	96.5308%	98.8380%
shaw	1,548	91	99.8515%	10.6555%	97.1741%	94.1214%
wardbeck	2,568	60	99.8688%	3.2058%	98.5512%	97.6636%
wheeler	4,584	141	99.9991%	10.3362%	99.8423%	96.9241%
wilson	4,586	54	99.9993%	4.5917%	99.9390%	98.8225%
Overall	42,355	709	99.9973%	3.7968%	99.8638%	98.3261%

Note: Input candidate objects are Alert or MetaAlert objects retained by Module 2 and submitted to Module 3. Object counts and ARR are measured at the candidate-object level, where $ARR = 1 - N_{\text{retained}}/N_{\text{input}}$. TPR, FPR, and F1 are measured at the member-alert level by expanding each Alert/MetaAlert decision to its constituent alerts entering Module 3. Specifically, $TPR = TP/(TP + FN)$ and $FPR = FP/(FP + TN)$.

reduction remains consistently high across all seven days, with the daily cumulative ARR ranging from 99.38% to 99.97%. On the remaining challenging candidates, Module 3 achieves an average TPR of 90.67%, FPR of 19.35%, and F1-score of 82.48%. These results demonstrate that the coarse-to-fine pipeline can substantially and consistently reduce analyst-facing workload while retaining effective alert adjudication under real-world, time-varying SOC workloads.

REFERENCES

- [1] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 99% false positives: A qualitative study of {SOC} analysts' perspectives on security alarms. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2783–2800, 2022.
- [2] Stefan Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security (TISSEC)*, 3(3):186–205, 2000.
- [3] Tao Ban, Ndichu Samuel, Takeshi Takahashi, and Daisuke Inoue. Combat security alert fatigue with ai-assisted techniques. In *Proceedings of the 14th Cyber Security Experimentation and Test Workshop*, pages 9–16, 2021.
- [4] Mohan Baruwal Chhetri, Shahroz Tariq, Ronal Singh, Fatemeh Jalalvand, Cecile Paris, and Surya Nepal. Towards human-ai teaming to mitigate alert fatigue in security operations centres. *ACM Transactions on Internet Technology*, 24(3):1–22, 2024.
- [5] Cisco. Snort. <https://www.snort.org/>, 2025.
- [6] CrowdStrike. Powering the next evolution of the soc. <https://www.crowdstrike.com/en-us/platform/charlotte-ai/>, 2025.
- [7] H. Debar, D. Curry, and B. Feinstein. The Intrusion Detection Message Exchange Format (IDMEF). RFC 4765, March 2007. URL: <https://www.rfc-editor.org/info/rfc4765/>, doi:10.17487/RFC4765.
- [8] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, 2024.
- [9] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Ryan Tsang, Najmeh Nazari, Han Wang, Houman Homayoun, et al. Large language models for code analysis: Do {LLMs} really do their job? In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 829–846, 2024.
- [10] Mohamed Amine Ferrag, Fatima Alwahedi, Ammar Battah, Bilel Cherif, Abdechakour Mechri, and Norbert Tihanyi. Generative ai and large language models for cyber security: All insights you need. *Available at SSRN 4853709*, 2024.
- [11] Scott Freitas and Amir Gharib. Graphweaver: Billion-scale cybersecurity incident correlation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4479–4486, 2024.
- [12] Yiru Gong, Song Liu, Changzhi Zhao, Junrong Liu, Tian Tian, Xiaobo Yang, Bo Jiang, and Zhigang Lu. Garnet: Got-based alert reduction and narrative event tracing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 327–335, 2026.
- [13] Google. Supercharging security with generative ai. <https://cloud.google.com/blog/products/identity-security/rsa-google-cloud-security-ai-workbench-generative-ai/>, 2023.
- [14] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. Nodoe: Combatting threat alert

TABLE X: Daily performance of SOCRATES on the Industry Arena Dataset during the seven-day test period.

Date	Total Alerts	Module 1				Module 2				Module 3		
		Type	Count	Agg. Total	ARR $\uparrow$	Type	Count	Agg. Total	ARR $\uparrow$	TPR $\uparrow$	FPR $\downarrow$	F1 $\uparrow$
Day1	273,824	Attack False Alarm	7,945 265,879	16,333	94.04%	Attack False Alarm	7,941 5,291	434	99.84%	93.90%	17.41%	84.38%
Day2	310,511	Attack False Alarm	9,347 301,164	18,092	94.17%	Attack False Alarm	9,194 7,767	1,912	99.38%	89.40%	19.08%	81.99%
Day3	318,788	Attack False Alarm	8,142 310,646	15,910	95.01%	Attack False Alarm	8,122 5,850	422	99.87%	90.12%	20.00%	81.11%
Day4	312,102	Attack False Alarm	8,291 303,811	16,003	94.87%	Attack False Alarm	8,286 6,344	226	99.93%	93.02%	20.00%	82.47%
Day5	273,395	Attack False Alarm	5,333 268,062	16,093	94.11%	Attack False Alarm	5,331 5,377	234	99.91%	92.04%	14.88%	88.51%
Day6	228,327	Attack False Alarm	5,743 222,584	12,826	94.38%	Attack False Alarm	5,741 5,073	75	99.97%	86.21%	17.39%	80.65%
Day7	221,778	Attack False Alarm	6,075 215,703	12,144	94.52%	Attack False Alarm	6,074 6,266	75	99.97%	90.00%	26.67%	78.26%
Total	1,938,725	/		107,401	94.46%	/		3,378	99.83%	90.67%	19.35%	82.48%

fatigue with automated provenance triage. In *network and distributed systems security symposium*, 2019.

- [15] Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. In *Findings of the association for computational linguistics: ACL 2023*, pages 1049–1065, 2023.
- [16] Sourov Jajodia, Madeena Sultana, Suryadipta Majumdar, Adrian Taylor, and Grant Vandenberghe. Before you hand over the wheel: Evaluating llms for security incident analysis. *arXiv preprint arXiv:2603.06422*, 2026.
- [17] Fatemeh Jalalvand, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2):1–36, 2024.
- [18] Hamed Jelodar, Samita Bai, Parisa Hamed, Hesamodin Mohammadian, Roozbeh Razavi-Far, and Ali Ghorbani. Large language model (llm) for software security: Code analysis, malware analysis, reverse engineering. *arXiv preprint arXiv:2504.07137*, 2025.
- [19] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. {ORTHRUS}: Achieving high quality of attribution in provenance-based intrusion detection systems. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 7173–7192, 2025.
- [20] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- [21] Max Landauer, Florian Skopik, Maximilian Frank, Wolfgang Hotwagner, Markus Wurzenberger, and Andreas Rauber. Maintainable log datasets for evaluation of intrusion detection systems. *IEEE Transactions on Dependable and Secure Computing*, 20(4):3466–3482, 2022.
- [22] Max Landauer, Florian Skopik, and Markus Wurzenberger. Introducing a new alert data set for multi-step attack analysis. In *Proceedings of the 17th cyber security experimentation and test workshop*, pages 41–53, 2024.
- [23] Xihuan Lin, Jie Zhang, Gelei Deng, Tianzhe Liu, Xiaolong Liu, Changcai Yang, Tianwei Zhang, Qing Guo, and Riqing Chen. Ircopilot: Automated incident response with large language models. *arXiv preprint arXiv:2505.20945*, 2025.
- [24] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhui Wang. Exploring {ChatGPT’s} capabilities on vulnerability management. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 811–828, 2024.
- [25] Samuel Ndichu, Tao Ban, Seiichi Ozawa, Takeshi Takahashi, and Daisuke Inoue. Nlog: Lightweight, explainable soc anomaly detection via log-to-language rewriting. *arXiv preprint arXiv:2606.04957*, 2026.
- [26] Samuel Ndichu, Tao Ban, Seiichi Ozawa, Takeshi Takahashi, and Daisuke Inoue. Pact: Reducing alert fatigue in low-prevalence soc streams with triggered active learning. *arXiv preprint arXiv:2605.22324*, 2026.
- [27] OISF. Suricata. <https://suricata.io/>, 2025.
- [28] Shuva Paul, Farhad Alemi, and Richard Macwan. Llm-assisted proactive threat intelligence for automated reasoning. *arXiv preprint arXiv:2504.00428*, 2025.
- [29] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2339–2356. IEEE, 2023.
- [30] OWASP Modsecurity Project. Modsecurity. <https://modsecurity.org/>, 2025.
- [31] Microsoft Security. Microsoft security copilot. <https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-security-copilot/>, 2025.
- [32] Mahmood Sharif, Pubali Datta, Andy Riddle, Kim Westfall, Adam Bates, Vijay Ganti, Matthew Lentzk, and David Ott. Drsec: Flexible distributed representations for efficient endpoint security. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3609–3624. IEEE, 2024.
- [33] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. Pentestagent: Incorporating llm agents to automated penetration testing. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 375–391, 2025.
- [34] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE symposium on security and privacy*, pages 305–316. IEEE, 2010.
- [35] Sanchana Srikanth, Mohammad Hasanuzzaman, and Farah Tasnur Meem. Evaluating the usability of llms in threat intelligence enrichment. *arXiv preprint arXiv:2409.15072*, 2024.
- [36] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. Alert fatigue in security operations centres: Research challenges and opportunities. *ACM Computing Surveys*, 57(9):1–38, 2025.

- [37] Ankit Thakkar and Ritika Lohiya. A review on machine learning and deep learning perspectives of ids for iot: Recent updates, security issues, and challenges. *Archives of computational methods in engineering*, 28(4), 2021.
- [38] Melissa Turcotte, François Labrèche, and Serge-Olivier Paquette. Automated alert classification and triage (aact): An intelligent system for the prioritisation of cybersecurity alerts. *arXiv preprint arXiv:2505.09843*, 2025.
- [39] Thijs Van Ede, Hojjat Aghakhani, Noah Spahn, Riccardo Bortolameotti, Marco Cova, Andrea Continella, Maarten Van Steen, Andreas Peter, Christopher Kruegel, and Giovanni Vigna. Deepcase: Semi-supervised contextual analysis of security events. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 522–539. IEEE, 2022.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [41] Mathew Vermeer, Michel Van Eeten, and Carlos Gañán. Ruling the rules: Quantifying the evolution of rulesets, alerts and incidents in network intrusion detection. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 799–814, 2022.
- [42] Bowen Wei, Yuan Shen Tay, Howard Liu, Jinhao Pan, Kun Luo, Ziwei Zhu, and Chris Jordan. Cortex: Collaborative llm agents for high-stakes alert triage. *arXiv preprint arXiv:2510.00311*, 2025.
- [43] Yiran Wu, Mauricio Velazco, Andrew Zhao, Manuel Raúl Meléndez Luján, Srisuma Movva, Yogesh K Roy, Quang Nguyen, Roberto Rodriguez, Qingyun Wu, Michael Albada, et al. Excytin-bench: Evaluating llm agents on cyber threat investigation. *arXiv preprint arXiv:2507.14201*, 2025.
- [44] HanXiang Xu, ShenAo Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Kai Chen, Ting Yu, Yang Liu, and HaoYu Wang. Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2024.
- [45] Limin Yang, Zhi Chen, Chenkai Wang, Zhenning Zhang, Sushruth Booma, Phuong Cao, Constantin Adam, Alexander Withers, Zbigniew Kalbarczyk, Ravishankar K Iyer, et al. True attacks, attack attempts, or benign triggers? an empirical measurement of network alerts in a security operations center. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1525–1542, 2024.
- [46] Xin Yin, Chao Ni, and Shaohua Wang. Multitask-based evaluation of open-source llm on software vulnerability. *IEEE Transactions on Software Engineering*, 50(11):3071–3087, 2024.
- [47] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.